

Can't I convert the c_k 's into actual polynomial coefficients in the original variable x and have an approximation of the following form?

$$f(x) \approx \sum_{k=0}^{m-1} g_k x^k, \quad a \leq x \leq b \quad (5.10.1)$$

Yes, you can do this (and we will give you the algorithm to do it), but we caution you against it: Evaluating equation (5.10.1), where the coefficient g 's reflect an underlying Chebyshev approximation, usually requires more significant figures than evaluation of the Chebyshev sum directly (as by `eval`). This is because the Chebyshev polynomials themselves exhibit a rather delicate cancellation: The leading coefficient of $T_n(x)$, for example, is 2^{n-1} ; other coefficients of $T_n(x)$ are even bigger; yet they all manage to combine into a polynomial that lies between ± 1 . Only when m is no larger than 7 or 8 should you contemplate writing a Chebyshev fit as a direct polynomial, and even in those cases you should be willing to tolerate two or so significant figures less accuracy than the roundoff limit of your machine.

You get the g 's in equation (5.10.1) in two steps. First, use the member function `polycofs` in `Chebyshev` to output a set of polynomial coefficients equivalent to the stored c_k 's (that is, with the range $[a, b]$ scaled to $[-1, 1]$). Second, use the routine `pcshft` to transform the coefficients so as to map the range back to $[a, b]$. The two required routines are listed here:

`chebyshev.h`

`VecDoub Chebyshev::polycofs(Int m)`

Polynomial coefficients from a Chebyshev fit. Given a coefficient array `c[0..n-1]`, this routine returns a coefficient array `d[0..n-1]` such that $\sum_{k=0}^{n-1} d_k y^k = \sum_{k=0}^{n-1} c_k T_k(y) - c_0/2$. The method is Clenshaw's recurrence (5.8.11), but now applied algebraically rather than arithmetically.

```
{
    Int k,j;
    Doub sv;
    VecDoub d(m),dd(m);
    for (j=0;j<m;j++) d[j]=dd[j]=0.0;
    d[0]=c[m-1];
    for (j=m-2;j>0;j--) {
        for (k=m-j;k>0;k--) {
            sv=d[k];
            d[k]=2.0*d[k-1]-dd[k];
            dd[k]=sv;
        }
        sv=d[0];
        d[0] = -dd[0]+c[j];
        dd[0]=sv;
    }
    for (j=m-1;j>0;j--) d[j]=d[j-1]-dd[j];
    d[0] = -dd[0]+0.5*c[0];
    return d;
}
```

`pcshft.h`

`void pcshft(Doub a, Doub b, VecDoub_I0 &d)`

Polynomial coefficient shift. Given a coefficient array `d[0..n-1]`, this routine generates a coefficient array `g[0..n-1]` such that $\sum_{k=0}^{n-1} d_k y^k = \sum_{k=0}^{n-1} g_k x^k$, where x and y are related by (5.8.10), i.e., the interval $-1 < y < 1$ is mapped to the interval $a < x < b$. The array `g` is returned in `d`.

```
{
    Int k,j,n=d.size();
    Doub cnst=2.0/(b-a), fac=cnst;
    for (j=1;j<n;j++) {
        d[j] *= fac;           First we rescale by the factor const...
        fac *= cnst;
    }
    cnst=0.5*(a+b);           ...which is then redefined as the desired shift.
```

```

for (j=0;j<=n-2;j++)
    for (k=n-2;k>=j;k--)
        d[k] -= cnst*d[k+1];
}

```

We accomplish the shift by synthetic division, a miracle of high-school algebra.

CITED REFERENCES AND FURTHER READING:

Acton, F.S. 1970, *Numerical Methods That Work*, 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 59, 182–183 [synthetic division].

5.11 Economization of Power Series

One particular application of Chebyshev methods, the *economization of power series*, is an occasionally useful technique, with a flavor of getting something for nothing.

Suppose that you know how to compute a function by the use of a convergent power series, for example,

$$f(x) \equiv \frac{1}{2} - \frac{x}{4} + \frac{x^2}{8} - \frac{x^3}{16} + \cdots \quad (5.11.1)$$

(This function is actually just $1/(x+2)$, but pretend you don't know that.) You might be doing a problem that requires evaluating the series many times in some particular interval, say $[0, 1]$. Everything is fine, except that the series requires a large number of terms before its error (approximated by the first neglected term, say) is tolerable. In our example, with $x = 1$, it takes about 30 terms before the first neglected term is $< 10^{-9}$.

Notice that because of the large exponent in x^{30} , the error is *much smaller* than 10^{-9} everywhere in the interval except at the very largest values of x . This is the feature that allows “economization”: If we are willing to let the error elsewhere in the interval rise to about the same value that the first neglected term has at the extreme end of the interval, then we can replace the 30-term series by one that is significantly shorter.

Here are the steps for doing this:

1. Compute enough coefficients of the power series to get accurate function values everywhere in the range of interest.
2. Change variables from x to y , as in equation (5.8.10), to map the x interval into $-1 \leq y \leq 1$.
3. Find the Chebyshev series (like equation 5.8.8) that exactly equals your truncated power series.
4. Truncate this Chebyshev series to a smaller number of terms, using the coefficient of the first neglected Chebyshev polynomial as an estimate of the error.
5. Convert back to a polynomial in y .
6. Change variables back to x .

We already have tools for all of the steps, except for steps 2 and 3. Step 2 is exactly the inverse of the routine `pcshft` (§5.10), which mapped a polynomial from y (in the interval $[-1, 1]$) to x (in the interval $[a, b]$). But since equation (5.8.10) is a linear relation between x and y , one can also use `pcshft` for the inverse. The inverse of

$$\text{pcshft}(a, b, d, n)$$

turns out to be (you can check this)

```

void ipcshft(Doub a, Doub b, VecDoub_IO &d) {
    pcshft(-(2.+b+a)/(b-a), (2.-b-a)/(b-a), d);
}

```

`pcshft.h`

Step 3 requires a new Chebyshev constructor, one that computes Chebyshev coefficients from a vector of polynomial coefficients. The following code accomplishes this. The algorithm is based on constructing the polynomial by the technique of §5.3 starting with the highest coefficient $d[n-1]$ and using the recurrence of equation (5.8.2) written in the form

$$\begin{aligned} xT_0 &= T_1 \\ xT_n &= \frac{1}{2}(T_{n+1} + T_{n-1}), \quad n \geq 1. \end{aligned} \quad (5.11.2)$$

The only subtlety is to multiply the coefficient of T_0 by 2 since it gets used with a factor 1/2 in equation (5.8.8).

chebyshev.h

```
Chebyshev::Chebyshev(VecDoub &d)
: n(d.size()), m(n), c(n), a(-1.), b(1.)
```

Inverse of routine `polycofs` in Chebyshev: Given an array of polynomial coefficients `d[0..n-1]`, construct an equivalent Chebyshev object.

```
{
    c[n-1]=d[n-1];
    c[n-2]=2.0*d[n-2];
    for (Int j=n-3;j>=0;j--) {
        c[j]=2.0*d[j]+c[j+2];
        for (Int i=j+1;i<n-2;i++) {
            c[i] = (c[i]+c[i+2])/2;
        }
        c[n-2] /= 2;
        c[n-1] /= 2;
    }
}
```

Putting them all together, steps 2 through 6 will look something like this (starting with a vector `powser` of power series coefficients):

```
ipcshft(a,b,powser);
Chebyshev cpowser(powser);
cpowser.setm(1.e-9);
VecDoub d=cpowser.polycofs();
pcshft(a,b,d);
```

In our example, by the way, the number of terms required for 10^{-9} accuracy is reduced from 30 to 9. Replacing a 30-term polynomial with a 9-term polynomial without any loss of accuracy — that does seem to be getting something for nothing. Is there some magic in this technique? Not really. The 30-term polynomial defined a function $f(x)$. Equivalent to economizing the series, we could instead have evaluated $f(x)$ at enough points to construct its Chebyshev approximation in the interval of interest, by the methods of §5.8. We would have obtained just the same lower-order polynomial. The principal lesson is that the rate of convergence of Chebyshev coefficients has nothing to do with the rate of convergence of power series coefficients; and it is the *former* that dictates the number of terms needed in a polynomial approximation. A function might have a *divergent* power series in some region of interest, but if the function itself is well-behaved, it will have perfectly good polynomial approximations. These can be found by the methods of §5.8, but *not* by economization of series. There is slightly less to economization of series than meets the eye.

CITED REFERENCES AND FURTHER READING:

Acton, F.S. 1970, *Numerical Methods That Work*, 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 12.

5.12 Padé Approximants

A *Padé approximant*, so called, is that rational function (of a specified order) whose power series expansion agrees with a given power series to the highest possible order. If the rational function is

$$R(x) \equiv \frac{\sum_{k=0}^M a_k x^k}{1 + \sum_{k=1}^N b_k x^k} \quad (5.12.1)$$

then $R(x)$ is said to be a Padé approximant to the series

$$f(x) \equiv \sum_{k=0}^{\infty} c_k x^k \quad (5.12.2)$$

if

$$R(0) = f(0) \quad (5.12.3)$$

and also

$$\left. \frac{d^k}{dx^k} R(x) \right|_{x=0} = \left. \frac{d^k}{dx^k} f(x) \right|_{x=0}, \quad k = 1, 2, \dots, M + N \quad (5.12.4)$$

Equations (5.12.3) and (5.12.4) furnish $M + N + 1$ equations for the unknowns $a_0, \dots, a_M$ and $b_1, \dots, b_N$. The easiest way to see what these equations are is to equate (5.12.1) and (5.12.2), multiply both by the denominator of equation (5.12.1), and equate all powers of x that have either a 's or b 's in their coefficients. If we consider only the special case of a diagonal rational approximation, $M = N$ (cf. §3.4), then we have $a_0 = c_0$, with the remaining a 's and b 's satisfying

$$\sum_{m=1}^N b_m c_{N-m+k} = -c_{N+k}, \quad k = 1, \dots, N \quad (5.12.5)$$

$$\sum_{m=0}^k b_m c_{k-m} = a_k, \quad k = 1, \dots, N \quad (5.12.6)$$

(note, in equation 5.12.1, that $b_0 = 1$). To solve these, start with equations (5.12.5), which are a set of linear equations for all the unknown b 's. Although the set is in the form of a Toeplitz matrix (compare equation 2.8.8), experience shows that the equations are frequently close to singular, so that one should not solve them by the methods of §2.8, but rather by full LU decomposition. Additionally, it is a good idea to refine the solution by iterative improvement (method `improve` in §2.5) [1].

Once the b 's are known, then equation (5.12.6) gives an explicit formula for the unknown a 's, completing the solution.

Padé approximants are typically used when there is some unknown underlying function $f(x)$. We suppose that you are able somehow to compute, perhaps by laborious analytic expansions, the values of $f(x)$ and a few of its derivatives at $x = 0$: $f(0)$, $f'(0)$, $f''(0)$, and so on. These are of course the first few coefficients in the power series expansion of $f(x)$; but they are not necessarily getting small, and you have no idea where (or whether) the power series is convergent.

By contrast with techniques like Chebyshev approximation (§5.8) or economization of power series (§5.11) that only condense the information that you already know about a function, Padé approximants can give you genuinely new information about your function's values.